Pruebas funcionales E2E

Metodología y ejecución — Ronda 3

Equipo SOS Clinic

22 de octubre de 2026

Resumen

Este documento describe cómo se planificaron y ejecutaron las pruebas funcionales de punta a punta de SOS Clinic sobre el stack local completo: autenticación con 2FA, gestión de pacientes, triage, signos vitales, alta con notificaciones, analítica, agente conversacional, auditoría y turnos. El objetivo es que cualquier integrante del equipo pueda reproducir la ejecución sin depender de conocimiento tácito.

Índice

1. Objetivo y alcance **1**

2. Entorno de ejecución **1**

3. Datos de prueba **2**

4. Diseño de las suites **2**

5. Procedimiento **2**

5.1. Consideraciones operativas 3

6. Criterios de aceptación **3**

7. Limitaciones **3**

8. Reproducción **3**

1 Objetivo y alcance

Se validó el comportamiento funcional observable por API a través del gateway (`http://127.0.0.1:8080`) y el frontend estático (`http://127.0.0.1:5500`). Quedaron fuera del alcance la automatización completa de la interfaz con Playwright y las pruebas de carga; el frontend se verificó con un smoke test de páginas y con la prueba de XSS en navegador real que se documenta en el informe de pentest.

2 Entorno de ejecución

Los servicios se levantaron con `04. levantar-sistema.bat`. Eureka registró los seis servicios y el gateway enroutó correctamente.

Componente	Versión / detalle
Sistema operativo	Windows 11, consola `cmd`
JDK	Eclipse Temurin 26.0.2
Maven	Wrapper 3.9.16 (por módulo)
Docker Desktop	29.9.2
Python	3.12
PostgreSQL	16, contenedor `sos-postgres`, puerto 5433
MySQL	servicio local, puerto 3306 (`urgencias_hospital`)
Eureka	8761
Gateway	8080
Seguridad / Pacientes / Triage	8081 / 8082 / 8083
Analítica / Agente IA	8084 / 8088
Frontend	5500 (servidor Python del repositorio)

Cuadro 1: Entorno usado en la fase funcional.

3 Datos de prueba

Se usó el seed canónico sin regenerarlo:

- 8 perfiles, 8 actividades y 22 asignaciones.
- 5 categorías de triage y 3 estados.
- 9 cuentas aprobadas con 2FA y turno de 24 horas.
- 9 turnos, 24 pacientes y 30 episodios.
- 0 códigos 2FA, 0 tokens y 0 registros de auditoría al inicio.

Las suites crean sus propios pacientes, episodios y cuentas temporales, y los eliminan al finalizar. Las credenciales se leen de `infra/seed/credenciales.local.env` y nunca se imprimen.

4 Diseño de las suites

Suite	Qué valida
`verify_seed.py`	Login completo con 2FA de las 9 cuentas del seed, perfil en la sesión y un endpoint representativo por rol.
`e2e_claves.py`	Cambio de clave, bloqueo por cambio pendiente (`ccp`), challenge 2FA de un solo uso, recuperación/activación, cambio de correo, aprobación y rechazo de cuentas, clave temporal y auditoría.
`e2e_funcional.py`	Frontend, pacientes, triage, signos vitales, alta orquestada con correos y PDF, reintento, analítica, agente IA, auditoría y turnos.
Maven	Pruebas unitarias y de contexto de los 7 módulos (documento aparte).

Cuadro 2: Suites ejecutadas en la fase funcional.

5 Procedimiento

1. **Preparación.** Se levantó Docker, PostgreSQL y el stack; se verificó el seed con `python infra/seed/verify_seed.py`. La base tenía dos residuos de pruebas previas (un código 2FA y una auditoría) que se eliminaron para dejar el conjunto canónico.
2. **Completar 2FA en pruebas.** Para no depender de la bandeja de correo, cada suite obtiene el `challenge` del login y reemplaza el código pendiente en la base por el hash BCrypt de

`123456` usando `crypt()` de PostgreSQL. Después llama a `/auth/verificar-2fa`. Esto no modifica el código de la aplicación.

3. **Claves y cuentas.** Se ejecutó `e2e_claves.py`: crea cuentas `e2e.*` con hashes de fixture, prueba los flujos y las borra.
4. **Flujo clínico.** Se ejecutó `e2e_funcional.py`: crea un paciente y dos episodios temporales, registra dos tomas de signos, da de alta un episodio con la llamada única de `finalizar`, descarga el PDF, reenvía correos y comprueba la idempotencia del alta.
5. **Frontend.** Se solicitan las 14 páginas y `assets/api.js` y se comprueba que respondan 200 con su contenido esperado.
6. **Limpieza.** Al final se eliminan pacientes, episodios, tomas, auditorías y cuentas temporales. La suite admite `-no-cleanup` para inspección manual.

5.1 Consideraciones operativas

- El limitador de reenvíos 2FA permite 3 códigos por usuario cada 10 minutos. Repetir la suite antes de esa ventana devuelve HTTP 429; se debe esperar o reiniciar `ms-seguridad` en local (el límite es en memoria).
- El alta envía correos reales a la bandeja de pruebas `pruebasosclinic@gmail.com`. No se usan datos clínicos reales.
- El agente IA consume la API de Groq; se hizo una consulta mínima.
- La analítica usa la base sintética `urgencias_hospital` con 50.000 llegadas.

6 Criterios de aceptación

Cada caso se registra como **PASS**, **FAIL**, **WARN** o **SKIP**. Un **FAIL** en una sección crítica detiene solo esa sección, no toda la suite. Se consideran críticos la creación del paciente, la creación del episodio, el alta persistida y la descarga del PDF. Los fallos de entrega SMTP se registran como **WARN** porque son una dependencia externa, siempre que el alta se haya persistido y el reintento esté disponible.

7 Limitaciones

- La cobertura es por API; la interfaz se valida con smoke test y una prueba de XSS, no con Playwright.
- No se midió cobertura de código (JaCoCo no está configurado).
- La entrega de correo y la respuesta del agente dependen de servicios externos (Gmail y Groq).
- El seed es compartido: las suites limpian sus datos, pero una interrupción abrupta puede dejar residuos.

8 Reproducción

```
04. levantar-sistema.bat
python infra/seed/verify_seed.py
python seguridad/pruebas/e2e_claves.py
python seguridad/pruebas/e2e_funcional.py
powershell -NoProfile -ExecutionPolicy Bypass ‘
  -File seguridad/pruebas/ejecutar-tests-maven.ps1
```